Answer Key and General Evaluation Guidelines

Subject: Advanced Web Technologies

Maximum Marks: 100

This answer key provides suggestive answers, expected technical points, sample SQL queries, diagrams, and programming solutions. Evaluators should award marks according to the correctness of the concepts, implementation, explanation, and output rather than requiring students to reproduce the exact wording or code shown here.

General Guidelines for Evaluators

1. Suggestive answers: The answers below are indicative, not exhaustive. Accept technically correct alternative explanations, examples, diagrams, programming languages, frameworks, and implementations consistent with the question.
2. Partial marking: Award marks for correct concepts, logical steps, syntax, and implementation even if the complete answer is incorrect or incomplete.
3. Programming questions: Evaluate logic, syntax, model/route definitions, database operations, and understanding. Minor syntax errors should not result in loss of all marks if the intended approach is clear.
4. Diagrams: Accept clearly labelled, logically correct diagrams. Artistic quality is not a criterion.
5. Equivalent terminology: Accept equivalent technical terminology where appropriate, such as ORM/ODM distinctions, database CRUD methods, and REST conventions.
6. No double penalty: Do not penalize the same underlying error repeatedly across related parts unless it independently affects multiple requirements.
7. Alternative answers: For Question 9, evaluate either option (a) or option (b), not both. The maximum remains 25 marks.
8. Marks allocation: Use the suggested mark distribution as a guide. Minor variations are acceptable where the total marks for each question remain unchanged.

Section A — 5 × 6 = 30 Marks

Q1. Explain session management and justify its need in web applications. (6 Marks)

Suggestive answer

Session management is the process of maintaining information about a user's interaction with a web application across multiple HTTP requests.

HTTP is stateless, meaning each request is independent by default. Session management allows the application to recognize a user and retain relevant information between requests.

Need for session management:

- Authentication: Maintains the authenticated user's state after login.
- Personalization: Stores preferences, language, or theme settings.
- Shopping carts: Maintains selected products across multiple pages.
- Access control: Helps restrict access to authorized users.
- Continuity: Preserves relevant application state during navigation.

Working mechanism:

1. A user logs in with valid credentials.
2. The server creates a session and generates a session identifier.
3. The identifier is commonly sent to the browser in a cookie.
4. The browser sends the cookie with subsequent requests.
5. The server identifies the session and retrieves the associated information.
6. The session is invalidated on logout or expiry.

Suggested marking scheme

Expected answer	Marks
Definition of session management	2
Explanation of session creation and identification	2
Any two valid needs or applications	2
Total	6

Evaluator note: Accept cookie-based sessions, server-side session stores, and correctly explained token-based approaches. Students need not describe all mechanisms.

Q2. Explain Create, Read, Update, and Delete (CRUD) operations with suitable examples. (6 Marks)

Suggestive answer

CRUD represents the four fundamental operations used to manage persistent data.

Operation	Purpose	Example
Create	Adds a new record	Register a new student
Read	Retrieves existing data	*Display student details*

Update	Modifies an existing record	Change a student's email
Delete	Removes a record	Delete a student record

Consider a student record with ID `101`, name `Amit`, and email `amit@example.com`.

- Create: Add Amit as a new student.
- Read: Retrieve the record with ID `101`.
- Update: Change the email address associated with ID `101`.
- Delete: Remove the student record with ID `101`.

CRUD operations may be implemented through SQL statements, REST API endpoints, or database libraries and ORMs.

Suggested marking scheme

Expected answer	Marks
Definition or explanation of CRUD	1
Create operation with example	1
Read operation with example	1
Update operation with example	1
Delete operation with example	1
Clear, relevant overall example	1
Total	6

Evaluator note: Accept examples using students, employees, products, or other appropriate entities.

Q3. Write SQL queries to demonstrate CRUD operations on a suitable table. (6 Marks)

Suggestive answer

Create a table named `Student`:

```sql
-- Create a table to store student records.
CREATE TABLE Student (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);

-- Create: Insert a new student.
INSERT INTO Student (id, name, email)
VALUES (101, 'Amit', 'amit@example.com');

-- Read: Retrieve all student records.
SELECT * FROM Student;

-- Update: Change the student's email.
UPDATE Student
SET email = 'amit.new@example.com'
WHERE id = 101;

-- Delete: Remove the student record.
DELETE FROM Student
WHERE id = 101;
```

Explanation:

- `INSERT` adds a record.
- `SELECT` retrieves records.
- `UPDATE` modifies records.

- `DELETE` removes records.
- `WHERE` restricts modifications or deletion to the intended record.

Suggested marking scheme

Expected answer	Marks
Correct `INSERT` query	1.5
Correct `SELECT` query	1
Correct `UPDATE` query	1.5
Correct `DELETE` query	1.5
Appropriate table or column references and explanation	0.5
Total	6

Evaluator note: Accept equivalent SQL syntax and valid alternative schemas. Deduct for a missing or incorrect `WHERE` clause where it makes an update or deletion unnecessarily affect all records.

Q4. Compare SQL and NoSQL databases based on data model, schema, scalability, and use cases. (6 Marks)

Suggestive answer

Basis	SQL databases	NoSQL databases
Data model	Primarily relational tables	Document, key-value, column-family, or graph models
Schema	Generally predefined and structured	Often flexible or model-dependent
Scalability	Commonly vertical scaling; horizontal scaling is also possible	Often designed to support horizontal scaling, depending on the database
Relationships	Supports relational operations and joins	Relationship handling depends on the data model; may use embedding, references, or graph traversal
Examples	MySQL, PostgreSQL, SQLite	MongoDB, Redis, Cassandra, Neo4j
Use cases	Banking, transactions, structured business data	Flexible documents, caching, large distributed workloads, graph relationships

SQL databases are often suitable when structured data, relational integrity, and complex transactions are important. NoSQL databases are useful when the chosen data model, flexible structure, or distributed architecture fits the application requirements.

Neither category is universally superior; the choice depends on the workload and consistency requirements.

Suggested marking scheme

Expected answer	Marks

Data model comparison	1.5
Schema comparison	1.5
Scalability comparison	1.5
Use cases and/or examples	1.5
Total	6

Evaluator note: Accept nuanced comparisons. Do not require students to claim that all SQL databases scale vertically or all NoSQL databases have flexible schemas.

Q5. Identify browser developer tools used to analyze HTTP requests and responses, and explain their use. (6 Marks)

Suggestive answer

Browser developer tools provide facilities for inspecting network communication between the browser and web server.

The Network panel in browsers such as Chrome, Firefox, and Edge is the principal tool for analyzing HTTP requests and responses.

Chrome DevTools | TopDev *(https://images.openai.com/static-rsc-4/iKUHcOloppg87bBavk07eRTRuIeoaq%3Cem%3E0l3btyYfgMDJt1WmhY5vN8Y5%3C/em%3E8AOZs469DcejHB15iXczlwGEpuv-7MNOlrDs1o0VwKxntVAPOhex_7P5wT3detL4zr7m0hUgOox0eyH8EgXoq5kC5mr77oLJuRdL-1HmTDhHUJEajtQ?purpose=inline)*

1\. Network panel

Lists HTTP requests, their URLs, methods, status codes, timing, and resource types. It helps identify failed or slow requests.

Chrome DevTools | Chrome for Developers *(https://images.openai.com/static-rsc-4/5sujR29pgso5q1-EPpN9mp69t69hyIfQNvGuXsRaryMT3Sx9hrx%3Cem%3EfRBuHeTrKqktyy-TskEJ05O2AxdugOIHVfbxaouNeWae3zu6Hb13ABAWBTUW002YQ7gxRCJRnOmD7a4C7n%3C/em%3EgOQDlATDSWCP9PJftlJCghD_LFycSBeu51gw?purpose=inline)*

2\. Headers and Payload

Shows request/response headers, HTTP methods, query parameters, request bodies, cookies, and content types.

How to extract data from this API? HttpRequest, JavaScript - Stack Overflow *(https://images.openai.com/static-rsc-4/Gp3Jtw4eooKDE6fL49kOdnbpNENOYdUcbs2ni8NYiZoYnjXAwEWOfOeifMGdZe2nxsJUWbjeWPmrHCKksOPtGqGBFOzYg1CSTByMfGmzgM6H9A-jASFUNVnBh_R2DC1iE231DXt6lXokPwKXLU4nsZFCtFSIYGz7GeXR1xxX7gg?purpose=inline)*

3\. Response and Preview

Displays the returned response body, including JSON, HTML, or error messages.

Additional useful features include:

- Status codes: Identify success, redirection, client errors, and server errors.
- Timing and Waterfall: Analyze latency and loading delays.
- Console: Inspect JavaScript errors that may affect network requests.
- Preserve log and Disable cache: Help investigate navigation-related requests and caching behavior.

Suggested marking scheme

Expected answer	Marks
Identification of Network panel	1
Explanation of request inspection	1.5
Explanation of response inspection	1.5
Status codes and timing analysis	1
One additional relevant tool or feature	1
Total	6

Evaluator note: Accept equivalent terminology and tools such as Firefox Network Monitor. Students are not required to name every subpanel.

Section B — 3 × 15 = 45 Marks

Q6. Explain the principles of RESTful APIs. Justify whether HTTP is stateless, using a suitable request-response example. (15 Marks)

Suggestive answer

REST (Representational State Transfer) is an architectural style for designing networked applications. A RESTful API exposes resources through identifiable URIs and uses standard HTTP methods to interact with them.

A. Principles of RESTful APIs (9 Marks)

1. Client-server separation: The client handles the user interface; the server handles data and business logic.
2. Stateless communication: Each request contains the information necessary to process it. The server does not depend on stored conversational context from previous requests.
3. Uniform interface: Resources are identified by URIs and accessed through consistent methods and representations.
4. Resource-based design: Resources such as students are represented by URIs, for example `/api/students/101`.
5. HTTP methods: `GET` retrieves data, `POST` creates or submits data, `PUT` replaces a resource, `PATCH` partially updates it, and `DELETE` removes it.
6. Cacheability: Responses should indicate whether they may be cached.
7. Layered system: Clients need not know whether they communicate directly with the application server or through intermediaries.
8. Representations: Resources can be represented in JSON, XML, or other suitable formats.
9. Code on demand (optional): A server may provide executable code to a client; this is an optional REST constraint.

B. Is HTTP stateless? (3 Marks)

HTTP is stateless at the protocol level. Each HTTP request is treated independently; HTTP itself does not automatically preserve application-session context between requests.

For example:

```
GET /api/students/101 HTTP/1.1
Host: example.com
Accept: application/json
```

The server processes the request and may return:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "id": 101,
  "name": "Amit"
}
```

A subsequent request must independently provide the information required to process it. The server does not automatically know the context of the earlier request merely because it occurred first.

C. Statelessness versus session management (3 Marks)

A web application can implement sessions using cookies or tokens, even though HTTP itself is stateless.

For example, an authenticated request might include:

```
GET /api/profile HTTP/1.1
Host: example.com
Cookie: sessionId=abc123
```

The server can use the cookie to identify a session. This does not make HTTP inherently stateful; it is an application-level mechanism built on top of HTTP.

In a strictly stateless REST interaction, the server should not depend on conversational session state to interpret the request.

Suggested marking scheme

Component	Marks
Explanation of REST and resource-oriented design	2
REST principles and constraints	7
Correct HTTP request-response example	3
Justification of HTTP statelessness	2
Distinction between HTTP statelessness and application sessions	1
Total	15

Evaluator note: Award full credit for a coherent explanation of the major REST constraints, even if the student does not list every constraint. The student should distinguish HTTP statelessness from the use of cookies, sessions, and tokens.

Q7. Analyze client-server architecture. Illustrate the roles of the client and server and their request-response interaction with a suitable diagram. (15 Marks)

Suggestive answer

Client-server architecture is a distributed computing model in which a client requests services or resources and a server processes requests and returns responses.

A. Roles of client and server (5 Marks)

Client:

- Provides the user interface.
- Accepts user input.
- Sends HTTP requests to the server.
- Processes returned data and displays results.
- May perform client-side validation and rendering.

Examples include web browsers, mobile applications, and frontend JavaScript applications.

Server:

- Accepts and processes client requests.
- Executes business logic and validation.
- Interacts with databases or external services.
- Applies authentication and authorization.
- Returns responses with status codes, headers, and content.

Examples include Node.js/Express, Django, and other web server applications.

B. Request-response diagram (5 Marks)

Client (Browser)

User interface, user input, JavaScript

HTTP Request

GET /api/students

Web/Application Server

Routing, validation, business logic

Database query / result

Database

Persistent application data

HTTP Response

200 OK + JSON data

The server sends the response to the client, which renders the result.

The diagram shows a common three-tier implementation. A valid two-tier client-server architecture without a separate database tier should also be accepted if explained correctly.

C. Request-response workflow (3 Marks)

1. The user performs an action in the browser.
2. The browser constructs and sends an HTTP request.
3. The server routes and validates the request.
4. The application performs the required logic and may query the database.
5. The server generates an HTTP response.
6. The browser processes the response and updates the interface.

D. Analysis: advantages and limitations (2 Marks)

Advantages:

- Centralized data and business logic.
- Easier maintenance and controlled access.
- Multiple clients can use the same backend services.

Limitations:

- Server downtime can affect multiple clients.
- Network latency affects response time.
- High request volumes may require load balancing and horizontal scaling.

Suggested marking scheme

Component	Marks
Definition and architecture explanation	2
Client and server roles	3
Correct, labelled diagram	5
Request-response workflow	3
Analysis of advantages and limitations	2
Total	15

Evaluator note: Accept diagrams using arrows, labelled boxes, or standard architecture notation. A separate database is not mandatory, but the roles of client and server must be clear.

Q8. Server-Side Rendering (SSR) (5 + 10 = 15 Marks)

Q8(a). Explain SSR and state two benefits. (5 Marks)

Suggestive answer

Server-Side Rendering (SSR) is a technique in which the server generates HTML for a requested page and sends the rendered HTML to the browser.

In SSR, the server processes the request, prepares the required data, renders the page, and returns HTML. The browser displays the returned page. JavaScript may subsequently add interactivity through hydration or other client-side code.

Benefits:

1. Improved initial rendering: The browser can display meaningful HTML before all client-side JavaScript has downloaded and executed.
2. Search engine optimization: Search engine crawlers can more easily access content already present in the HTML response, although indexing depends on the crawler and application.

Other acceptable benefits include improved initial content visibility, accessibility, and support for clients with limited JavaScript execution capabilities.

Suggested marking scheme

Expected answer	Marks

Correct definition of SSR	2
Explanation of server-generated HTML and browser response	1
First valid benefit	1
Second valid benefit	1
Total	5

Q8(b). Write a program to demonstrate SSR using Node.js or Python. (10 Marks)

The following is one acceptable solution using Node.js and Express.

Program: `server.js`

```
// Import Express to create the web server.
const express = require("express");

const app = express();
const PORT = 3000;

// Sample data that the server will render into HTML.
const student = {
    id: 101,
    name: "Amit",
    course: "BCA"
};

// Generate HTML on the server for the requested page.
app.get("/", (req, res) => {
    const html = `
        <!DOCTYPE html>
        <html lang="en">
        <head>
            <meta charset="UTF-8">
            <title>Student Details</title>
        </head>
        <body>
            <h1>Student Details</h1>
            <p>ID: ${student.id}</p>
            <p>Name: ${student.name}</p>
            <p>Course: ${student.course}</p>
        </body>
        </html>
    `;

    // Send the server-generated HTML to the browser.
    res.status(200).type("html").send(html);
});

// Start the HTTP server.
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Execution:

```
# Initialize the project and install Express.
npm init -y
npm install express

# Start the server.
node server.js
```

Open `http://localhost:3000` in a browser. The browser receives HTML generated by the server, displaying the student's details.

Why this demonstrates SSR: The server constructs the HTML using student data before sending it to the browser. The browser does not need to build the entire page from a client-side JavaScript data structure.

A Python implementation using Flask and a server-side template engine such as Jinja2 is equally acceptable.

Suggested marking scheme

Expected answer	Marks
Correct server framework setup and imports	2
Route definition and request handling	2
Dynamic HTML generation or template rendering	3
Correct HTTP response containing HTML	1
Execution instructions, output, or explanation of SSR	2
Total	10

Evaluator note: Accept a correct Flask/Jinja2 implementation. Full marks should not depend on using Express specifically. A static HTML file merely served by the server, without server-side rendering of the relevant content, should receive limited credit.

Section C — 1 × 25 = 25 Marks

Q9(a). Student Enrollment System: ER Diagram, Database Models, ORM and CRUD (10 + 15 = 25 Marks)

Part A: ER diagram and database models (10 Marks)

Suggestive answer

A Student Enrollment System records students, courses, and the enrollment of students in courses.

A suitable relational design uses three entities:

- Student: Stores student details.
- Course: Stores course details.
- Enrollment: Connects students with the courses in which they enroll.

A student may enroll in multiple courses, and each course may have multiple students. This is a many-to-many relationship implemented through the Enrollment entity.

ER diagram

STUDENT

```
PK: student_id
name
email
```

COURSE

```
PK: course_id
course_name
credits
1 : M
student_id is a foreign key
```

ENROLLMENT

```
PK: enrollment_id
FK: student_id
```

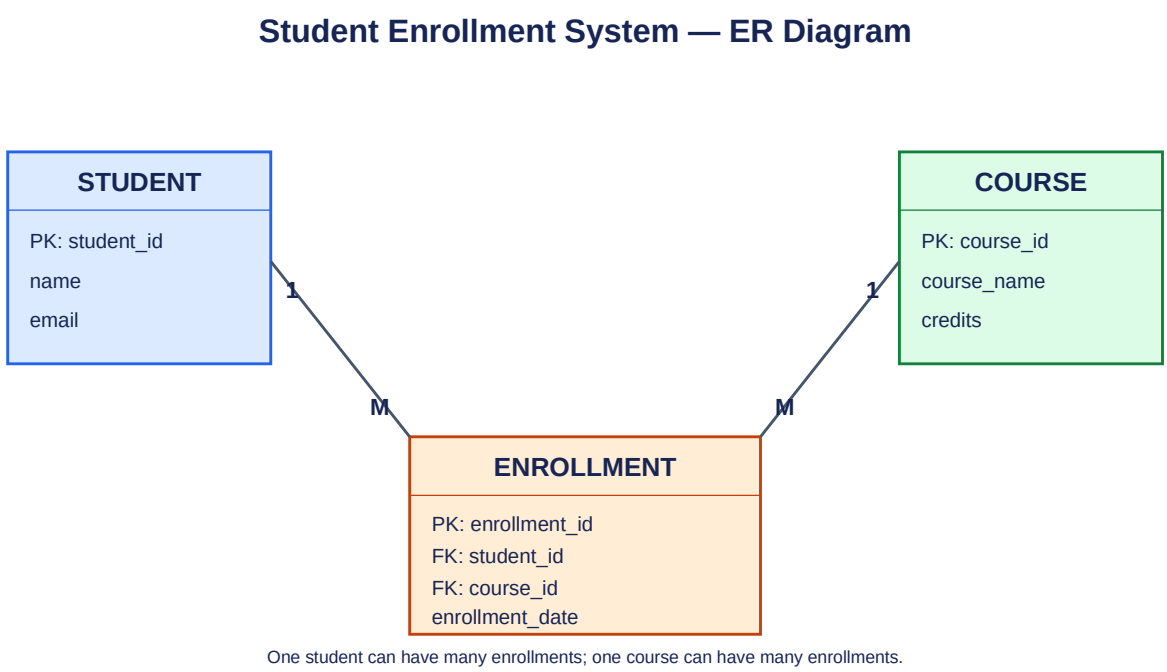
```
FK: course_id

enrollment_date

M : 1

course_id is a foreign key
```

Together, Student and Course have a many-to-many relationship through Enrollment.



The diagram is a conceptual representation. Students may use conventional ER notation, such as crow's-foot cardinalities, instead of this layout.

Explanation of entities, attributes, keys and cardinalities

- Entities: Student, Course, Enrollment.
- Attributes: Student name/email; course name/credits; enrollment date.
- Primary keys (PK): Uniquely identify records in each entity.
- Foreign keys (FK): `studentid` *and* `course id` in Enrollment reference their corresponding entities.
- Cardinality: One student can have many enrollment records, and one course can have many enrollment records.
- Referential integrity: Foreign keys prevent enrollment records from referencing nonexistent students or courses.

Brief explanation of database models

Accept any suitable set of database models with accurate explanations. For example:

Model	Description
Relational	Data is organized into tables with rows, columns, and relationships.
Document	Data is stored in documents, commonly JSON-like structures.
Key-value	Data is stored as key-value pairs.
Graph	Data is represented through nodes and relationships or edges.
Hierarchical	Data is organized in a tree-like parent-child structure.
Network	Records can be connected through more general graph-like links.

Suggested marking scheme — Part A

Expected answer	Marks
Identification of suitable entities	2
Attributes and primary keys	2
Foreign keys and relationships	2
Correct cardinalities and explanation	2
Explanation of database models	2
Total	10

Evaluator note: Accept equivalent ER designs. For example, a composite primary key `(studentid, courseid)` in Enrollment is also valid if duplicate enrollment in the same course is not intended. Additional attributes are acceptable.

Part B: Define a Student model using an ORM and perform CRUD operations (15 Marks)

One acceptable implementation uses Django ORM with SQLite.

1. Define the model

In `models.py` :

```
from django.db import models# Django maps this class to a database table.class Student(models.Model):    name = models.Cha
```

2. Create and apply migrations

```
# Generate the database migration from the model.
python manage.py makemigrations

# Create or update the database tables.
python manage.py migrate
```

3. Perform CRUD operations

The following code can be run in the Django shell using `python manage.py shell` .

```
from yourapp.models import Student# Create: Insert a new student.student = Student.objects.create(    name="Amit",    emai
```

Replace `yourapp` with the actual Django application name.

Suggested marking scheme — Part B

Expected answer	Marks
Correct ORM/framework setup or imports	2
Student model with appropriate fields	3
Create operation	2
Read operation	2
Update operation	2
Delete operation	2
Migration/execution instructions or explanation	2
Total	15

Evaluator note: Accept equivalent implementations using SQLAlchemy, Sequelize, or another suitable ORM. The student must demonstrate the model definition and all four CRUD operations. Award partial marks when the framework is correctly identified but one or more operations are incomplete.

Q9(b). Express routes returning JSON and HTML, plus MongoDB CRUD using Mongoose (25 Marks)

This is an alternative to Q9(a). Evaluate only the option attempted by the student.

Part A: Express routes returning JSON and HTML (10 Marks)

Suggestive answer

The following Express application defines two routes:

- `/students/json` returns student data in JSON format.
- `/students/html` returns the same data as an HTML page.

Program: `server.js`

```
const express = require("express");

const app = express();
const PORT = 3000;

// Sample data used by both routes.
const students = [
    { id: 1, name: "Amit", course: "BCA" },
    { id: 2, name: "Neha", course: "MCA" }
];

// Route 1: Return student data as JSON.
app.get("/students/json", (req, res) => {
    res.status(200).json(students);
});

// Route 2: Generate and return an HTML page.
app.get("/students/html", (req, res) => {
    const studentRows = students.map(student => `
        <tr>
            <td>${student.id}</td>
            <td>${student.name}</td>
            <td>${student.course}</td>
        </tr>
    `).join("");

    // Construct the HTML response on the server.
    const html = `
        <!DOCTYPE html>
        <html lang="en">
        <head>
            <meta charset="UTF-8">
            <title>Students</title>
        </head>
        <body>
            <h1>Student List</h1>
            <table border="1">
                <tr><th>ID</th><th>Name</th><th>Course</th></tr>
                ${studentRows}
            </table>
        </body>
        </html>
    `;

    res.status(200).type("html").send(html);
});

// Start the server.
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Execution:

```
npm init -y
npm install express
node server.js
```

Test both routes in a browser or API client:

- `http://localhost:3000/students/json`

- `http://localhost:3000/students/html`

Suggested marking scheme — Express routes (10 Marks)

Expected answer	Marks
Correct Express initialization and server setup	2
Correct JSON route and response	3
Correct HTML route and response	3
Route accessibility, execution, or explanation	2
Total	10

Evaluator note: Accept routes using alternative paths. The essential requirement is that one route returns JSON and the other returns HTML.

Part B: Connect to MongoDB, define a Student model using Mongoose, and perform CRUD (15 Marks)

1. Install dependencies

```
npm install express mongoose
```

Ensure a MongoDB server is available and the connection URI is correct.

2. Define the model and connect to MongoDB

The following example can be placed in `student-db.js`.

```
const mongoose = require("mongoose");

// Connect to the MongoDB database.
async function connectDB() {
    await mongoose.connect("mongodb://127.0.0.1:27017/studentDB");
    console.log("Connected to MongoDB");
}

// Define the structure and validation rules for students.
const studentSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true,
        unique: true
    },
    course: {
        type: String,
        required: true
    }
});

// Create a Mongoose model for database operations.
const Student = mongoose.model("Student", studentSchema);
```

3. Perform CRUD operations

Append the following code to `student-db.js`:

```
async function performCRUD() {
    try {
        // Connect before accessing the database.
        await connectDB();

        // Create: Insert a student document.
        const student = await Student.create({
            name: "Amit",
            email: "amit@example.com",
            course: "BCA"
        });

        // Read: Retrieve the inserted student.
        const foundStudent = await Student.findById(student._id);
```

```
        console.log("Read:", foundStudent);

        // Update: Change the student's course.
        const updatedStudent = await Student.findByIdAndUpdate(
            student._id,
            { course: "MCA" },
            { new: true, runValidators: true }
        );
        console.log("Updated:", updatedStudent);

        // Delete: Remove the student document.
        await Student.findByIdAndDelete(student._id);
        console.log("Student deleted");
    } catch (error) {
        console.error("Database operation failed:", error.message);
    } finally {
        // Close the connection after the demonstration.
        await mongoose.disconnect();
    }
}

// Execute the CRUD demonstration.
performCRUD();
```

Run the program using:

```
node student-db.js
```

Explanation:

- Mongoose is an Object Data Modeling (ODM) library for MongoDB.
- A schema defines document fields and validation rules.
- A model provides methods for interacting with the MongoDB collection.
- `create()` inserts a document.
- `findById()` retrieves a document.
- `findByIdAndUpdate()` updates a document.
- `findByIdAndDelete()` deletes a document.

Suggested marking scheme — Mongoose (15 Marks)

Expected answer	Marks
MongoDB connection using Mongoose	3
Student schema and model definition	3
Create operation	2
Read operation	2
Update operation	2
Delete operation	2
Error handling or execution explanation	1
Total	15

Evaluator note: Accept equivalent Mongoose methods such as `save()`, `find()`, `updateOne()`, and `deleteOne()`. Do not require the exact URI or field names used above. Award credit for a correct connection strategy, model definition, and CRUD logic even if the student's environment-specific configuration is incomplete.

Consolidated Marks Summary

Question	Maximum marks
Q1. Session management	6

Q2. CRUD concepts	6
Q3. SQL CRUD	6
Q4. SQL vs NoSQL	6
Q5. Browser developer tools	6
Q6. RESTful APIs and HTTP statelessness	15
Q7. Client-server architecture	15
Q8(a). SSR concepts	5
Q8(b). SSR program	10
Q9(a) OR Q9(b)	25
Total	100

Final evaluation checklist

Before finalizing marks, confirm that:

- The marks awarded for each question do not exceed its maximum.
- Question 9 is evaluated for only one option, either (a) or (b).
- Programming answers receive partial credit for correct logic and relevant concepts.
- Equivalent technologies and technically valid alternative solutions are accepted.
- The total of Sections A, B, and C is 30 + 45 + 25 = 100 marks.